Kubernetes Task – 3

Environment Details

Component	Configuration
Kubernetes Master	Ubuntu EC2 – master
Worker Nodes	worker-01, worker-02
Kubernetes Version	v1.34.13
Container Runtime	containerd
CNI Plugin	Flannel
Working Directory	~/kubernetes-tasks
Namespace	k8s-session-3

All tasks were performed from the Kubernetes master node.

1. Create a ClusterIP service for an Apache web server pod.

Objective

Create an Apache Pod and expose it internally using a ClusterIP Service.

Step 1: Create the working directory

mkdir -p ~/kubernetes-tasks

cd ~/kubernetes-tasks

Step 2: Create the namespace

kubectl create namespace k8s-session-3

Verify:

kubectl get namespaces

Step 3: Create the Apache Pod YAML file

vi apache-pod.yml

Add the following configuration:

```
apiVersion: v1
kind: Pod
metadata:
 name: apache-pod
 namespace: k8s-session-3
 labels:
  app: apache
spec:
 containers:
  - name: apache-container
    image: httpd:2.4
    ports:
     - containerPort: 80
```

Save the file and apply it:

kubectl apply -f apache-pod.yml

Verify the Pod:

kubectl get pods -n k8s-session-3 -o wide

Expected result: Apache Pod should be in Running status.

Step 4: Create the ClusterIP Service YAML file

vi apache-service.yml

Add:

```
apiVersion: v1
kind: Service
metadata:
```

```
  name: apache-service
  namespace: k8s-session-3
spec:
  type: ClusterIP
  selector:
    app: apache
  ports:
    - port: 80
      targetPort: 80
```

Apply the configuration:

```
kubectl apply -f apache-service.yml
```

Step 5: Verify the Service

```
kubectl get svc -n k8s-session-3
```

Check the Service endpoints:

```
kubectl get endpoints apache-service -n k8s-session-3
```

The endpoint should show the Apache Pod IP on port 80.

Step 6: Test Apache connectivity internally

Create a temporary curl Pod:

```
kubectl run curl-test \
  --image=curlimages/curl:8.12.1 \
  --restart=Never \
  -n k8s-session-3 \
  -it --rm -- sh
```

Inside the container, execute:

```
curl http://apache-service
```

Expected result: Apache HTML response containing It works!.

Exit the container:

```
exit
```

Result

Apache Pod was successfully created and exposed internally through a ClusterIP Service. Connectivity was verified using curl from inside the Kubernetes cluster.

```
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Fri Nov  6 10:36:21 2026 from 103.203.172.230
ubuntu@master:~$ kubectl get nodes
NAME        STATUS   ROLES           AGE     VERSION
master      Ready    control-plane   4d20h   v1.34.13
worker-01   Ready    <none>          4d19h   v1.34.13
worker-02   Ready    <none>          4d19h   v1.34.13
ubuntu@master:~$ kubectl create namespace k8s-session-3
namespace/k8s-session-3 created
ubuntu@master:~$ kubectl get namespaces
NAME              STATUS   AGE
default           Active   4d20h
k8s-session-3     Active   4s
kube-flannel      Active   4d19h
kube-node-lease   Active   4d20h
kube-public       Active   4d20h
kube-system       Active   4d20h
```

```
ubuntu@master:~$ ls
kubernetes-tasks
ubuntu@master:~$ cd ~/kubernetes-tasks
pwd
/home/ubuntu/kubernetes-tasks
ubuntu@master:~/kubernetes-tasks$ vi apache-pod.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f apache-pod.yml
pod/apache-pod created
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -n k8s-session-3 -o wide
NAME         READY   STATUS    RESTARTS   AGE   IP           NODE        NOMINATED NODE   READINESS GATES
apache-pod   1/1     Running   0          38s   10.244.2.10   worker-02   <none>           <none>
ubuntu@master:~/kubernetes-tasks$
```

```
ubuntu@master:~/kubernetes-tasks$ vi apache-service.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f apache-service.yml
service/apache-service created
ubuntu@master:~/kubernetes-tasks$ kubectl get services -n k8s-session-3
NAME             TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
apache-service   ClusterIP   10.108.244.241   <none>        80/TCP    9s
ubuntu@master:~/kubernetes-tasks$
```

```
ubuntu@master:~/kubernetes-tasks$ kubectl get endpoints apache-service -n k8s-session-3
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME             ENDPOINTS          AGE
apache-service   10.244.2.10:80   40s
ubuntu@master:~/kubernetes-tasks$ kubectl run curl-test \
  --image=curlimages/curl:8.12.1 \
  --restart=Never \
  -n k8s-session-3 \
  -it --rm \
  -- sh
All commands and output from this session will be recorded in container logs, including credentials and sensitive information passed through the command prompt.
If you don't see a command prompt, try pressing enter.
~ $ curl http://apache-service
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
<title>It works! Apache httpd</title>
</head>
<body>
<p>It works!</p>
</body>
</html>
~ $ exit
pod "curl-test" deleted from k8s-session-3 namespace
ubuntu@master:~/kubernetes-tasks$ kubectl get all -n k8s-session-3
NAME             READY   STATUS    RESTARTS   AGE
pod/apache-pod   1/1     Running   0          3m51s

NAME                     TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)   AGE
service/apache-service   ClusterIP   10.108.244.241   <none>        80/TCP    111s
ubuntu@master:~/kubernetes-tasks$
```

2. Expose an Nginx pod externally using a NodePort service.

Objective

Create an Nginx Pod and expose it externally using a NodePort Service.

Step 1: Create the Nginx Pod YAML file

vi nginx-nodeport-pod.yml

Add:

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-nodeport-pod
  namespace: k8s-session-3
  labels:
    app: nginx-nodeport
spec:
  containers:
    - name: nginx-container
      image: nginx:latest
      ports:
        - containerPort: 80
```

Apply:

```
kubectl apply -f nginx-nodeport-pod.yml
```

Verify:

```
kubectl get pods -n k8s-session-3 -o wide
```

Step 2: Create the NodePort Service YAML file

```
vi nginx-nodeport-service.yml
```

Add:

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport-service
  namespace: k8s-session-3
spec:
  type: NodePort
```

```
  selector:
    app: nginx-nodeport
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

Apply:

kubectl apply -f nginx-nodeport-service.yml

Step 3: Verify the Service

kubectl get svc -n k8s-session-3

Verify endpoints:

kubectl get endpoints nginx-nodeport-service -n k8s-session-3

The Service should use NodePort 30080 and point to the Nginx Pod on port 80.

Step 4: Access Nginx externally

1. Open AWS Management Console.
2. Navigate to EC2 → Instances.
3. Select either worker node.
4. Copy its Public IPv4 address.
5. Ensure its Security Group allows inbound TCP port 30080 from your IP.
6. Open the following URL in a browser:

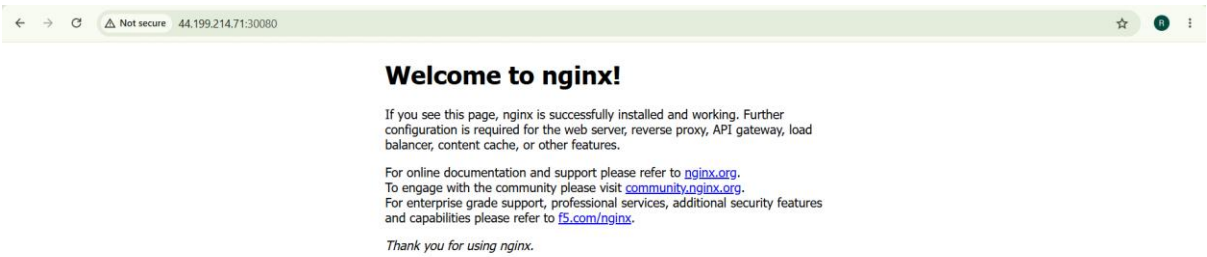
http://<WORKER-PUBLIC-IP>:30080

Expected result: Nginx Welcome page.

Result

Nginx Pod was successfully exposed using a NodePort Service. The Service configuration and internal endpoint were verified. External browser access can be recorded after completing the browser test.

```
ubuntu@master:~/kubernetes-tasks$ vi nginx-nodeport-pod.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f nginx-nodeport-pod.yml
pod/nginx-nodeport-pod created
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -n k8s-session-3 -o wide
NAME                READY   STATUS    RESTARTS   AGE     IP            NODE        NOMINATED NODE   READINESS GATES
apache-pod          1/1     Running   0          5m39s   10.244.2.10   worker-02   <none>           <none>
nginx-nodeport-pod  1/1     Running   0          7s      10.244.2.12   worker-02   <none>           <none>
ubuntu@master:~/kubernetes-tasks$ vi nginx-nodeport-service.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f nginx-nodeport-service.yml
service/nginx-nodeport-service created
ubuntu@master:~/kubernetes-tasks$ kubectl get svc -n k8s-session-3
NAME                    TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)        AGE
apache-service          ClusterIP   10.108.244.241   <none>        80/TCP         5m19s
nginx-nodeport-service  NodePort    10.102.240.26    <none>        80:30080/TCP   18s
ubuntu@master:~/kubernetes-tasks$ kubectl get endpoints nginx-nodeport-service -n k8s-session-3
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME                    ENDPOINTS         AGE
nginx-nodeport-service  10.244.2.12:80    2m34s
ubuntu@master:~/kubernetes-tasks$
```



3. Deploy a ReplicationController to maintain 3 replicas of an Nginx pod.

Objective

Create a ReplicationController that maintains three Nginx Pod replicas.

Step 1: Create the ReplicationController YAML file

vi nginx-rc.yml

Add:

apiVersion: v1

kind: ReplicationController

metadata:

 name: nginx-rc

 namespace: k8s-session-3

spec:

 replicas: 3

 selector:

  app: nginx-rc

```
 template:
  metadata:
   labels:
    app: nginx-rc
  spec:
   containers:
    - name: nginx-container
      image: nginx:latest
      ports:
       - containerPort: 80
```

Step 2: Apply the configuration

kubectl apply -f nginx-rc.yml

Step 3: Verify the ReplicationController

kubectl get rc -n k8s-session-3

Expected result:

- Desired: 3
- Current: 3
- Ready: 3

Check the Pods:

kubectl get pods -n k8s-session-3 -l app=nginx-rc -o wide

Three Nginx Pods should be running.

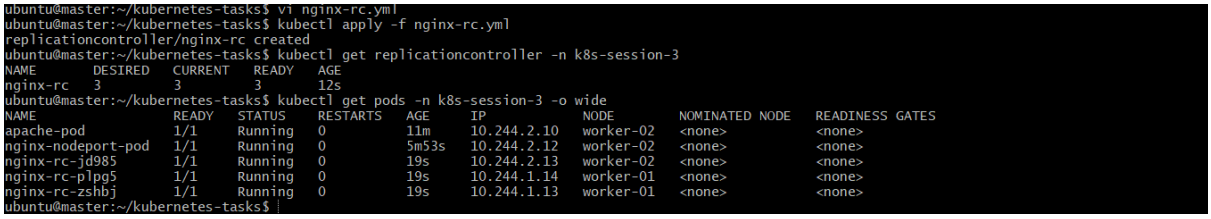
Step 4: Understand the working

The ReplicationController continuously monitors the number of matching Pods.

If one Pod is deleted or fails, the ReplicationController creates another Pod to maintain the desired replica count.

Result

A ReplicationController was successfully created to maintain three Nginx Pod replicas.



4. Scale the ReplicationController from 3 replicas to 5 replicas.

Objective

Increase the number of Nginx Pod replicas managed by the existing ReplicationController from three to five.

Step 1: Open the existing YAML file

vi nginx-rc.yml

Initially, the configuration contained:

replicas: 3

Change it to:

replicas: 5

The updated section:

spec:

 replicas: 5

 selector:

  app: nginx-rc

Step 2: Apply the updated configuration

kubectl apply -f nginx-rc.yml

Step 3: Verify the scaling operation

kubectl get rc -n k8s-session-3

Expected output should show:

NAME DESIRED CURRENT READY

nginx-rc 5 5 5

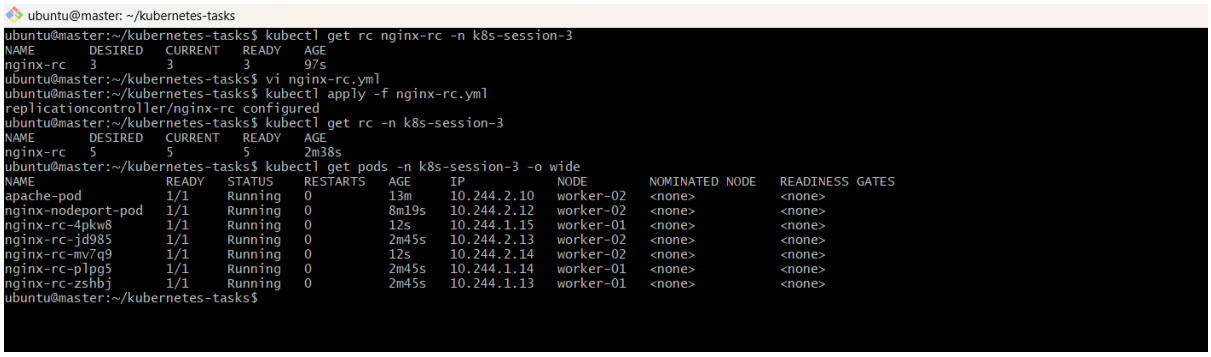
Check all five Pods:

kubectl get pods -n k8s-session-3 -l app=nginx-rc -o wide

Five Nginx Pods should be running across the worker nodes.

Result

The ReplicationController was successfully scaled from three to five replicas by updating the replicas field in the YAML configuration.



5. Create a ReplicaSet to manage pods based on multiple labels (prod and test).

Objective

Create a ReplicaSet using matchExpressions to select Pods whose environment label has either prod or test as its value.

Step 1: Create the ReplicaSet YAML file

vi nginx-rs-multilabel.yml

Add:

apiVersion: apps/v1

kind: ReplicaSet

metadata:

name: nginx-rs-multilabel

namespace: k8s-session-3

```
spec:
 replicas: 3
 selector:
  matchExpressions:
   - key: environment
     operator: In
     values:
      - prod
      - test
 template:
  metadata:
   labels:
    environment: prod
    app: nginx-multilabel
  spec:
   containers:
    - name: nginx-container
      image: nginx:latest
      ports:
       - containerPort: 80
```

Step 2: Apply the configuration

kubectl apply -f nginx-rs-multilabel.yml

Step 3: Verify the ReplicaSet

kubectl get rs -n k8s-session-3

Check the Pods:

kubectl get pods -n k8s-session-3 -l environment=prod

Three Nginx Pods should be running with the environment=prod label.

Step 4: Understand matchExpressions

The selector uses:

operator: In

values:

- prod

- test

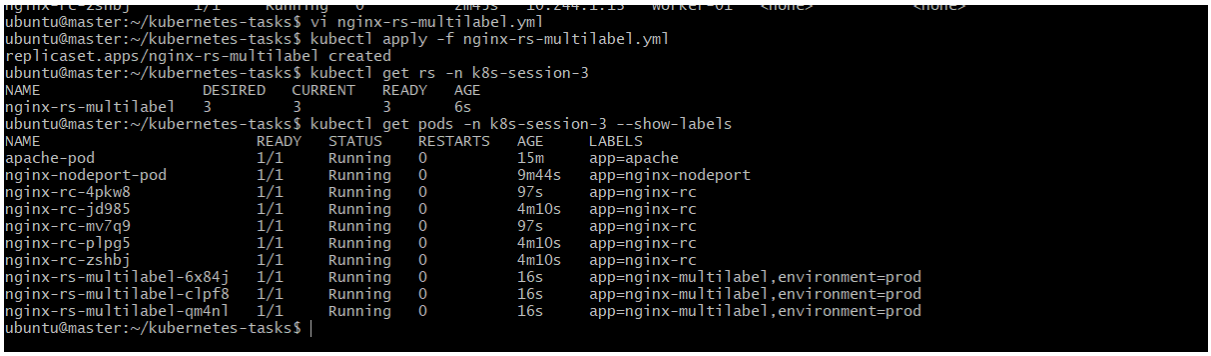
This means the ReplicaSet selects Pods where the environment label is either prod OR test.

In this task, the Pod template assigned environment: prod, so all three created Pods had the prod label.

Note: The selector supports both values, but this task created only prod-labelled Pods.

Result

A ReplicaSet was successfully created using matchExpressions with the In operator to select Pods based on multiple permitted label values.



6. Deploy a ReplicaSet that excludes pods with the label backend.

Objective

Create a ReplicaSet that selects Pods with an environment label, but excludes Pods whose environment value is backend.

Step 1: Create the ReplicaSet YAML file

vi nginx-rs-exclude.yml

Add:

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs-exclude
  namespace: k8s-session-3
spec:
  replicas: 3
  selector:
    matchExpressions:
      - key: environment
        operator: Exists
      - key: environment
        operator: NotIn
        values:
          - backend
  template:
    metadata:
      labels:
        environment: frontend
        app: nginx-exclude
    spec:
      containers:
        - name: nginx-container
          image: nginx:latest
          ports:
            - containerPort: 80
```

Step 2: Apply the configuration

```
kubectl apply -f nginx-rs-exclude.yml
```

Step 3: Verify the ReplicaSet

kubectl get rs nginx-rs-exclude -n k8s-session-3

Check the Pods:

kubectl get pods -n k8s-session-3 -l app=nginx-exclude

Expected result: Three Nginx Pods should be running.

Step 4: Understand the selector

The ReplicaSet uses two expressions:

1. Exists

- key: environment

  operator: Exists

The Pod must have an environment label.

2. NotIn

- key: environment

  operator: NotIn

  values:

    - backend

The Pod's environment value must not be backend.

Both conditions must be satisfied.

The Exists condition is important because NotIn alone can also match Pods that do not have the specified label.

Important implementation note

During the task, we initially created the ReplicaSet without the Exists condition. Its selector also matched the temporary curl Pod because that Pod had no environment label.

We corrected the selector. Since ReplicaSet selectors are immutable after creation, we deleted the old ReplicaSet while preserving its Pods:

kubectl delete rs nginx-rs-exclude -n k8s-session-3 --cascade=orphan

Then we applied the corrected YAML again:

kubectl apply -f nginx-rs-exclude.yml

The corrected ReplicaSet successfully maintained three running Pods.

Result

A ReplicaSet was successfully created using Exists and NotIn expressions to select labelled Pods while excluding the backend environment.

```
ubuntu@master:~/kubernetes-tasks$ vi nginx-rs-exclude.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f nginx-rs-exclude.yml
replicaset.apps/nginx-rs-exclude created
ubuntu@master:~/kubernetes-tasks$ kubectl get rs -n k8s-session-3
NAME                DESIRED   CURRENT   READY   AGE
nginx-rs-exclude    3         3         3       9s
nginx-rs-multilabel 3         3         3       119s
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -n k8s-session-3 --show-labels
NAME                        READY   STATUS    RESTARTS   AGE     LABELS
apache-pod                  1/1     Running   0          17m     app=apache
nginx-nodeport-pod          1/1     Running   0          11m     app=nginx-nodeport
nginx-rc-4pkw8              1/1     Running   0          3m31s   app=nginx-rc
nginx-rc-jd985              1/1     Running   0          6m4s    app=nginx-rc
nginx-rc-mv7q9              1/1     Running   0          3m31s   app=nginx-rc
nginx-rc-plpg5              1/1     Running   0          6m4s    app=nginx-rc
nginx-rc-zshbj              1/1     Running   0          6m4s    app=nginx-rc
nginx-rs-exclude-zns8t      1/1     Running   0          20s     app=nginx-exclude,environment=frontend
nginx-rs-multilabel-6x84j   1/1     Running   0          2m10s   app=nginx-multilabel,environment=prod
nginx-rs-multilabel-clpf8   1/1     Running   0          2m10s   app=nginx-multilabel,environment=prod
nginx-rs-multilabel-qm4nl   1/1     Running   0          2m10s   app=nginx-multilabel,environment=prod
ubuntu@master:~/kubernetes-tasks$
```

7. Test load balancing across multiple pods using a NodePort service.

Objective

Create multiple Nginx Pods and expose them through a NodePort Service. Verify that requests are distributed across different backend Pods.

Step 1: Create an Nginx Deployment YAML file

vi nginx-lb-deployment.yml

Add:

apiVersion: apps/v1

kind: Deployment

metadata:

 name: nginx-lb-deployment

 namespace: k8s-session-3

spec:

 replicas: 3

 selector:

```
  matchLabels:
    app: nginx-lb
 template:
  metadata:
    labels:
      app: nginx-lb
  spec:
    containers:
      - name: nginx-container
        image: nginx:latest
        ports:
          - containerPort: 80
```

Step 2: Apply the Deployment

kubectl apply -f nginx-lb-deployment.yml

Verify:

kubectl get deployment -n k8s-session-3

Check the three Pods:

kubectl get pods -n k8s-session-3 -l app=nginx-lb -o wide

Three Nginx Pods should be running.

Step 3: Create the NodePort Service YAML file

vi nginx-lb-service.yml

Add:

```
apiVersion: v1
kind: Service
metadata:
 name: nginx-lb-service
 namespace: k8s-session-3
spec:
```

```
  type: NodePort
  selector:
    app: nginx-lb
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30081
```

Step 4: Apply the Service

kubectl apply -f nginx-lb-service.yml

Verify:

kubectl get svc nginx-lb-service -n k8s-session-3

Check the backend endpoints:

kubectl get endpoints nginx-lb-service -n k8s-session-3

Expected result: Three Pod IP addresses should appear, each using port 80.

Step 5: Create a temporary curl Pod

```
kubectl run curl-lb-test \
  --image=curlimages/curl:8.12.1 \
  --restart=Never \
  -n k8s-session-3 \
  --command -- sleep 3600
```

Verify:

kubectl get pod curl-lb-test -n k8s-session-3

The Pod should show Running status.

Step 6: Configure unique responses for each Nginx Pod

To identify which Pod handles each request, update the default Nginx page on each backend Pod.

Execute:

```
for pod in $(kubectl get pods -n k8s-session-3 -l app=nginx-lb -o name); do
```

```
  kubectl exec -n k8s-session-3 "$pod" -- sh -c 'echo "Served by: $(hostname)" > /usr/share/nginx/html/index.html'
done
```

Verify the Pods:

```
kubectl get pods -n k8s-session-3 -l app=nginx-lb -o wide
```

Step 7: Test load balancing from inside the cluster

Enter the curl Pod:

```
kubectl exec -it curl-lb-test -n k8s-session-3 -- sh
```

Execute multiple requests:

```
for i in $(seq 1 15); do
  curl -s http://nginx-lb-service
done
```

Expected output will contain different backend Pod hostnames, for example:

```
Served by: nginx-lb-deployment-7cb4946fbf-8rl7p
Served by: nginx-lb-deployment-7cb4946fbf-vtk58
Served by: nginx-lb-deployment-7cb4946fbf-8jgmd
```

In our practical, all three backend Pods responded to the 15 requests.

Exit the container:

```
exit
```

Step 8: External NodePort verification

The Service can also be accessed externally using a worker node's public IP:

```
http://<WORKER-PUBLIC-IP>:30081
```

Ensure the worker node's EC2 Security Group allows inbound TCP port 30081 from your IP.

Result

A Deployment with three Nginx replicas was created and exposed through a NodePort Service. The Service endpoints were verified, and internal curl testing confirmed that requests reached all three backend Pods.

```
ubuntu@master:~/kubernetes-tasks$ vi nginx-rs-exclude.yml
ubuntu@master:~/kubernetes-tasks$ kubectl apply -f nginx-rs-exclude.yml
replicaset.apps/nginx-rs-exclude created
ubuntu@master:~/kubernetes-tasks$ kubectl get rs nginx-rs-exclude -n k8s-session-3
NAME              DESIRED   CURRENT   READY   AGE
nginx-rs-exclude   3         3         3       6s
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -n k8s-session-3 -l app=nginx-exclude
NAME                     READY   STATUS    RESTARTS   AGE
nginx-rs-exclude-2b422   1/1     Running   0          12s
nginx-rs-exclude-2q4qn   1/1     Running   0          12s
nginx-rs-exclude-zns8t   1/1     Running   0          13m
ubuntu@master:~/kubernetes-tasks$ kubectl run curl-lb-test \
  --image=curlimages/curl:8.12.1 \
  --restart=Never \
  -n k8s-session-3 \
  --command -- sleep 3600
pod/curl-lb-test created
ubuntu@master:~/kubernetes-tasks$ kubectl get pod curl-lb-test -n k8s-session-3
NAME          READY   STATUS    RESTARTS   AGE
curl-lb-test   1/1     Running   0          5s
ubuntu@master:~/kubernetes-tasks$
```

```
ubuntu@master:~/kubernetes-tasks$ kubectl get svc nginx-lb-service -n k8s-session-3
kubectl get endpoints nginx-lb-service -n k8s-session-3
NAME               TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
nginx-lb-service   NodePort   10.110.40.141   <none>        80:30081/TCP   11m
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME               ENDPOINTS                                               AGE
nginx-lb-service   10.244.1.18:80,10.244.2.17:80,10.244.2.18:80   11m
ubuntu@master:~/kubernetes-tasks$
```

```
nginx-lb-service   NodePort   10.110.40.141   <none>        80:30081/TCP   11m
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME               ENDPOINTS                                               AGE
nginx-lb-service   10.244.1.18:80,10.244.2.17:80,10.244.2.18:80   11m
ubuntu@master:~/kubernetes-tasks$ for pod in $(kubectl get pods -n k8s-session-3 -l app=nginx-lb -o name); do
  kubectl exec -n k8s-session-3 "$pod" -- sh -c 'echo "Served by: $(hostname)" > /usr/share/nginx/html/index.html'
done
ubuntu@master:~/kubernetes-tasks$ kubectl get pods -n k8s-session-3 -l app=nginx-lb -o wide
NAME                                  READY   STATUS    RESTARTS   AGE   IP            NODE        NOMINATED NODE   READINESS GATES
nginx-lb-deployment-7cb4946fbf-8jgmd   1/1     Running   0          13m   10.244.2.17   worker-02   <none>           <none>
nginx-lb-deployment-7cb4946fbf-8rl7p   1/1     Running   0          13m   10.244.2.18   worker-02   <none>           <none>
nginx-lb-deployment-7cb4946fbf-vtk58   1/1     Running   0          13m   10.244.1.18   worker-01   <none>           <none>
ubuntu@master:~/kubernetes-tasks$ kubectl exec -it curl-lb-test -n k8s-session-3 -- sh
~ $ for i in $(seq 1 15); do
>   curl -s http://nginx-lb-service
> done
Served by: nginx-lb-deployment-7cb4946fbf-8rl7p
Served by: nginx-lb-deployment-7cb4946fbf-vtk58
Served by: nginx-lb-deployment-7cb4946fbf-8rl7p
Served by: nginx-lb-deployment-7cb4946fbf-8jgmd
Served by: nginx-lb-deployment-7cb4946fbf-vtk58
Served by: nginx-lb-deployment-7cb4946fbf-8jgmd
Served by: nginx-lb-deployment-7cb4946fbf-8jgmd
Served by: nginx-lb-deployment-7cb4946fbf-8jgmd
Served by: nginx-lb-deployment-7cb4946fbf-vtk58
Served by: nginx-lb-deployment-7cb4946fbf-8rl7p
Served by: nginx-lb-deployment-7cb4946fbf-8jgmd
Served by: nginx-lb-deployment-7cb4946fbf-vtk58
Served by: nginx-lb-deployment-7cb4946fbf-8rl7p
Served by: nginx-lb-deployment-7cb4946fbf-8jgmd
Served by: nginx-lb-deployment-7cb4946fbf-vtk58
~ $ exit
ubuntu@master:~/kubernetes-tasks$ kubectl get svc nginx-lb-service -n k8s-session-3
NAME               TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
nginx-lb-service   NodePort   10.110.40.141   <none>        80:30081/TCP   13m
ubuntu@master:~/kubernetes-tasks$
```



Served by: nginx-lb-deployment-7cb4946fbf-vtk58

8. Delete a ReplicationController without affecting the running pods.

Objective

Delete the ReplicationController while keeping its existing five Nginx Pods running.

Step 1: Verify the existing ReplicationController

kubectl get rc -n k8s-session-3

Before deletion, the nginx-rc ReplicationController was maintaining five replicas.

Step 2: Delete the ReplicationController using orphan propagation

Execute:

kubectl delete replicationcontroller nginx-rc -n k8s-session-3 --cascade=orphan

Expected output:

replicationcontroller "nginx-rc" deleted

Step 3: Verify that the ReplicationController is deleted

kubectl get rc nginx-rc -n k8s-session-3

Expected result:

Error from server (NotFound): replicationcontrollers "nginx-rc" not found

Step 4: Verify that the Pods are still running

kubectl get pods -n k8s-session-3 -l app=nginx-rc

Expected result: All five Nginx Pods should remain in Running status.

In our practical, the five Pods were:

nginx-rc-4pkw8

nginx-rc-jd985

nginx-rc-mv7q9

nginx-rc-plpg5

nginx-rc-zshbj

All five remained running after deleting the ReplicationController.

Step 5: Understand orphan deletion

The --cascade=orphan option deletes the controller but preserves its dependent Pods.

Normal deletion	**Orphan deletion**
Deletes the controller	Deletes the controller
Normally deletes dependent Pods	Preserves dependent Pods

Uses cascading deletion by default Uses orphan propagation

Result

The ReplicationController was successfully deleted without deleting its five dependent Nginx Pods.

